ZUKUNFTSTAG 2026

JUNIOR EMBEDDED DEVELOPER

Mein Workbook: Vom ersten LED-Licht zum selbstfahrenden Roboter



GRUNDLAGEN

Was ist Embedded Software?

Software steckt nicht nur in Computern.

Embedded bedeutet „eingebettet“: Ein kleiner Computer ist in ein Gerät eingebaut und übernimmt dort bestimmte Aufgaben. Er liest Signale, trifft Entscheidungen und steuert etwas.

Beispiele aus dem Alltag

Waschmaschine: misst Temperatur und steuert den Motor.

Fahrradlicht: reagiert auf einen Sensor.

Auto: steuert Fensterheber, Beleuchtung oder Assistenzfunktionen.

Was macht ein Embedded-Entwickler?

Er entwickelt Software für Geräte: Hardware verstehen, Programme schreiben, ausprobieren und Fehler suchen. Dabei arbeitet er oft mit anderen Fachleuten.

Unser Ziel heute: Erst eine LED-Anzeige programmieren, dann Tasten und Sensoren benutzen und schliesslich einen Roboter fahren lassen.

GRUNDLAGEN

Der Mikrocontroller

Ein winziger Computer für eine konkrete Aufgabe.

CPU und Speicher

Die CPU führt Befehle aus. Im Flash-Speicher liegt das Programm. Im RAM merkt sich das Gerät Werte, während es läuft.

Eingänge: Woher kommen Informationen?

Taster, Licht- und Bewegungssensoren liefern Signale. Ein Programm kann diese Signale abfragen und darauf reagieren.

Ausgänge: Was kann der Controller steuern?

LEDs, Lautsprecher, Displays und Motoren. Für stärkere Verbraucher wie Motoren braucht man einen geeigneten Treiber.

Wichtig: Ein Mikrocontroller ist nicht dasselbe wie ein PC. Er hat meist weniger Speicher und Rechenleistung, ist dafür kompakt und stromsparend.

HARDWARE

Unser BBC micro:bit V2

Das Entwicklungsboard, mit dem wir beginnen.

Vorderseite

25 rote LEDs als 5 × 5-Matrix, zwei Tasten A und B und ein berührungsempfindliches Logo. Damit kann man sofort kleine Programme ausprobieren.

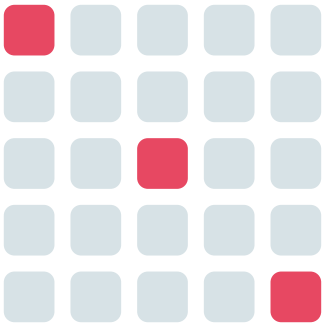
Weitere Hardware

Bewegungssensor, Kompass, Mikrofon, Lautsprecher und Funk. Über USB wird das Programm übertragen; mit Batterien funktioniert das Board mobil.

Anschlüsse

Am unteren Rand liegen Kontakte für Stromversorgung und Signale. GPIO bedeutet „General Purpose Input/Output“ – vielseitig verwendbare Ein- und Ausgänge.

LED-Koordinaten: links oben (0,0), rechts unten (4,4).

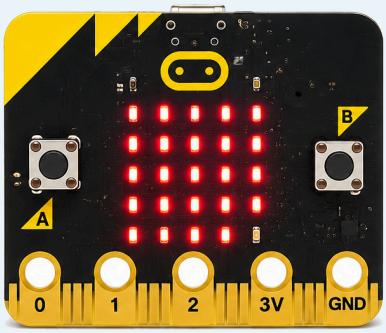


HARDWARE

Unsere beiden Geräte

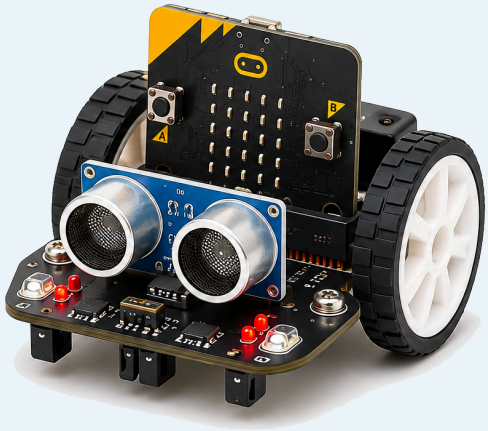
Vom ersten LED-Muster zum selbstfahrenden Roboter.

BBC micro:bit V2



25 LEDs | 2 Tasten | Sensoren

Maqueen Lite V5



Motoren | Linie | Ultraschall

micro:bit: Wir programmieren zuerst die LED-Matrix und Tasten.

Maqueen: Danach stecken wir den micro:bit in den Roboter.

PROGRAMMIEREN

Was ist eine Programmiersprache?

Ein Ziel – verschiedene Wege zum gleichen Ergebnis.

MakeCode-Blöcke

Farbige Blöcke zusammensetzen. Ideal zum Einstieg.

JavaScript in MakeCode

Eine textbasierte Sprache. MakeCode zeigt Blöcke auch als Code.

MicroPython

Python für Mikrocontroller. Befehle werden als Text geschrieben.

C und C++

Wichtige Sprachen für professionelle Embedded-Software.

Das gleiche Herz – drei Darstellungen

1 MakeCode-Blöcke

beim Start → zeige Symbol ♥

2 MakeCode-JavaScript

```
basic.showIcon(IconNames.Heart)
```

3 MicroPython

```
from microbit import *
display.show(Image.HEART)
```

MakeCode stellt seine micro:bit-Funktionen bereits bereit. In MicroPython importieren wir die Funktionen mit «from microbit import *».

MAKECODE

Programmieren mit Blöcken

Die wichtigste Programmierumgebung für unseren Zukunftstag.

HIER STARTEN

makecode.microbit.org

Kostenlos im Browser – keine Installation oder Anmeldung nötig.

So findest du dich zurecht

1 Simulator

Virtueller micro:bit: Programm direkt ausprobieren.

2 Blockkategorien

Hier findest du Grundlagen, Eingabe, Logik und Variablen.

3 Arbeitsbereich

Blöcke zusammenstecken; «beim Start» und «dauerhaft».

4 Herunterladen

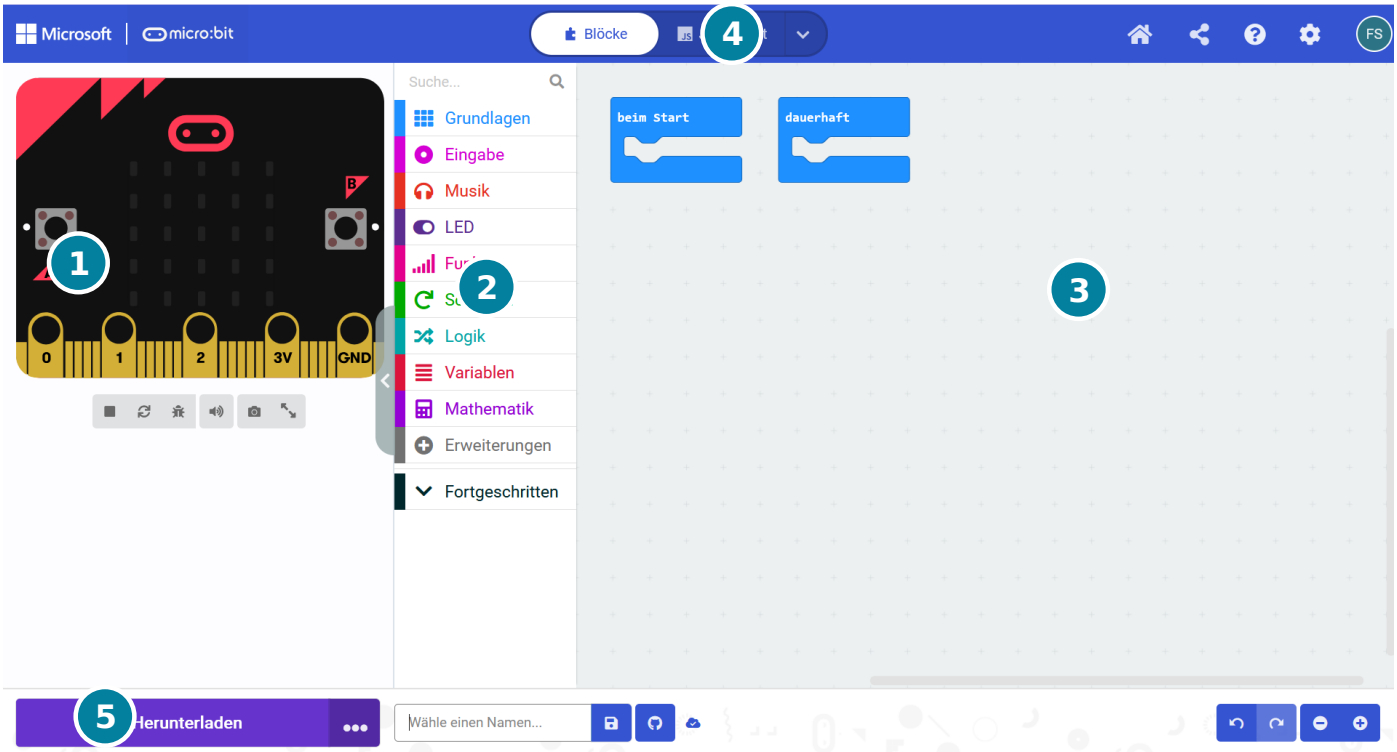
Programm per USB auf den micro:bit übertragen.

Deine Projekte speichern

Du kannst ohne Anmeldung sofort programmieren. Die Projekte bleiben dann normalerweise im Browser dieses Computers gespeichert.
Möchtest du später zu Hause weiterarbeiten, melde dich am besten mit deinem Microsoft-Konto (z. B. Hotmail) an. So können deine Projekte über die Cloud auf anderen Computern verfügbar sein.

Die Oberfläche auf einen Blick

So sieht der echte MakeCode-Editor aus.



1 Simulator:
Links kannst du dein Programm ausprobieren.

2 Blockkategorien:
Hier findest du LEDs, Tasten, Schleifen und Variablen.

3 Programmierbereich:
Ziehe die Blöcke hierher und verbinde sie.

4 Blöcke / JavaScript:
Oben kannst du zwischen Blöcken und Code wechseln.

5 Herunterladen:
Unten links überträgst du das Programm auf den micro:bit.

Die wichtigsten Bausteine

Mit wenigen Konzepten entstehen bereits tolle Programme.

Befehl

„zeige Symbol“ zeigt ein Bild auf der LED-Matrix.

Ereignis

„wenn Knopf A geklickt“ startet Code nach einem Tastendruck.

Bedingung

„wenn ... dann ... sonst“ entscheidet anhand einer Prüfung.

Schleife

„dauerhaft“ wiederholt Befehle, solange das Programm läuft.

Variable

Eine Variable speichert einen Wert, z. B. einen Punktestand.

Pause

„pausiere (ms)“ wartet eine bestimmte Anzahl Millisekunden.

PRAXIS

Deine micro:bit-Aufgaben

Arbeite in MakeCode mit den farbigen Blöcken.

1. Ein Herz anzeigen

Zeige beim Start ein Herz auf der LED-Matrix.

MakeCode-Tipp

Grundlagen · beim Start; Grundlagen · zeige Symbol

☐ **Geschafft!**

2. Eigene Pixel

Schalte die mittlere LED (2, 2) ein. Ergänze weitere Punkte und zeichne ein eigenes Muster.

MakeCode-Tipp

LED: zeichne Punkt bei x/y; x und y: 0 bis 4

☐ **Geschafft!**

3. Zwei Tasten, zwei Gesichter

Taste A soll einen glücklichen Smiley zeigen, Taste B einen traurigen.

MakeCode-Tipp

Eingabe: wenn Knopf geklickt; Grundlagen: zeige Symbol

☐ **Geschafft!**

PRAXIS

Aufgabe 4: Lauflicht ohne Schleife

25 LEDs. Ein Lichtpunkt. Du bestimmst den Weg.

Deine Aufgabe

Lass einen einzelnen Lichtpunkt über das gesamte 5 × 5-Display wandern. Starte links oben bei (0, 0) und ende rechts unten bei (4, 4). Jede LED soll kurz leuchten und danach wieder ausgehen.

So gehst du vor

1. Erstelle ein neues MakeCode-Projekt.
2. Beginne mit den ersten drei Positionen.
3. Ergänze danach alle weiteren Positionen bis zur letzten LED.
4. Teste, ob immer nur eine LED gleichzeitig leuchtet.

TIPP | Drei Blöcke für jede Position

LED → zeichne Punkt bei x/y
Grundlagen → pausiere (ms)
LED → lösche Punkt bei x/y

Denk nach

Welche Koordinate kommt nach (4, 0)?

Wie viele einzelne LED-Positionen musst du programmieren?

Was passiert, wenn du das Löschen einer LED vergisst?

PRAXIS

Aufgabe 5: Lauflicht mit Schleife

Gleicher Effekt – mit zwei verschachtelten Schleifen.

Deine Aufgabe

Lass eine einzelne LED über alle 25 Positionen wandern. Verwende diesmal zwei Schleifen: eine für die Zeilen (y) und eine für die Spalten (x). Das Lauflicht soll sich wiederholen.

So gehst du vor

1. Erstelle ein neues MakeCode-Projekt.
2. Setze eine für-Schleife in eine zweite für-Schleife.
3. Lass beide Schleifen von 0 bis 4 zählen.
4. Schalte jede LED kurz ein und danach wieder aus.

TIPP | Zwei Schleifen

Die äussere Schleife wählt die Zeile y. Die innere Schleife läuft durch die Spalten x. Beide verwenden den Block «für … von 0 bis 4».

Zum Knobeln

Welche Schleife muss innen stehen, damit das Licht zeilenweise wandert?

Warum werden mit zwei Schleifen genau 25 LEDs erreicht?

Freiwillige Herausforderung: Schlangenlauflicht

Lass das Licht in jeder zweiten Zeile in die andere Richtung laufen.

Tipp: Unter «Mathematik» findest du den Block «Rest von …».

Was verrät dir der Rest bei geraden und ungeraden Zeilen?

Wie kannst du die X-Koordinate in jeder zweiten Zeile spiegeln?

PRAXIS

Zusatzaufgabe: Tempo per Tastendruck

Erweitere dein Lauflicht: Du bestimmst die Geschwindigkeit!

Deine Aufgabe

Dein Lauflicht soll weiterhin über das ganze Display wandern.

Jetzt kannst du das Tempo mit den Tasten verändern:

Taste A: schneller **Taste B: langsamer**

Die Animation soll auch nach mehreren Tastendrücken funktionieren.

So gehst du vor

1. Verwende eine Variable für die Wartezeit zwischen den LEDs.
2. Setze die Wartezeit zu Beginn auf 100 Millisekunden.
3. Nutze die Ereignisse für Taste A und Taste B.
4. Verändere die Wartezeit bei jedem Tastendruck um 10 ms.
5. Verwende deine Variable im Pause-Block des Lauflichts.

TIPP | Welche Blöcke brauchst du?

Eingabe → wenn Knopf A gedrückt

Eingabe → wenn Knopf B gedrückt

Variablen → ändere ... um ...

Grundlagen → pausiere (ms) ...

Zum Knobeln

Was passiert, wenn die Wartezeit 0 oder sogar negativ wird?

Wie kannst du verhindern, dass sie unter 10 ms oder über 2000 ms steigt?

Bonus: Setze mit A+B das Tempo wieder auf 100 ms zurück.

Programmieren mit Text

Optional: Nach den MakeCode-Blöcken kannst du Python ausprobieren.

OFFIZIELLER MICROPYTHON-EDITOR

python.microbit.org

MicroPython ist eine Python-Variante für Mikrocontroller.
Hier schreibst du Befehle als Text statt Blöcke zu verbinden.

Beispiel: Taste A zeigt einen glücklichen Smiley

```
from microbit import *

while True:
    if button_a.is_pressed():
        display.show(Image.HAPPY)
    else:
        display.show(Image.SAD)
    sleep(100)
```

Was bedeutet der Code?

from microbit import *	Stellt micro:bit-Funktionen bereit.
while True:	Wiederholt den eingerückten Programmteil.
if / else	Entscheidet zwischen zwei Möglichkeiten.
sleep(100)	Wartet 100 Millisekunden.

Merke: In Python sind die Einrückungen wichtig!
Sie zeigen, welche Befehle zusammengehören.

Unser Maqueen Lite V5

Aus einem Programm wird ein fahrender Roboter.

Aktoren und Sensoren

Zwei Motoren bewegen die Räder. Die Liniensensoren erkennen helle und dunkle Flächen.
Der Ultraschallsensor misst Abstände. RGB-LEDs sorgen für Licht.

Die Idee des Linienfolgers

Liegt die schwarze Linie in der Mitte, fährt der Roboter geradeaus. Erkennt er die Linie links, korrigiert er nach links; rechts entsprechend nach rechts.

So beginnen wir

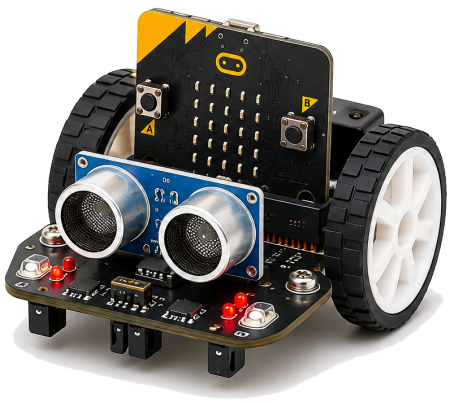
Zuerst Motoren langsam vorwärtsfahren lassen. Danach die Sensorwerte prüfen. Erst dann die Regeln zum Lenken ergänzen und auf einer breiten schwarzen Linie testen.

Zum Schluss: ausprobieren, beobachten, Geschwindigkeit anpassen und Fehler verbessern. Genau so läuft echte Softwareentwicklung!

Deine Aufgabe: Maqueen selbst in Betrieb nehmen

Öffne das offizielle DFRobot-Wiki und folge dem Getting Started.
Baue den Roboter auf, teste die ersten Funktionen und beobachte, was passiert.

https://wiki.dfrobot.com/mbt0046/docs/21343



SPICKZETTEL

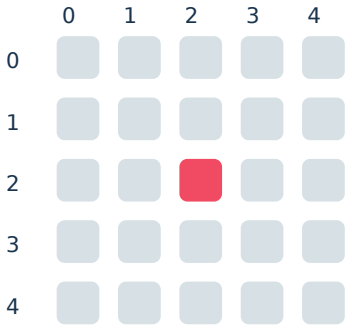
Deine Programmierhilfe

Zum Nachschauen während des Zukunftstags.

MakeCode: makecode.microbit.org

MicroPython: python.microbit.org

LED-Matrix: x → und y ↓ (jeweils 0 bis 4)



Mitte: (2, 2)

1000 ms = 1 Sekunde

MakeCode: Wo finde ich was?

Grundlagen	Symbole, Text, Pausen, Bildschirm löschen
Eingabe	Tasten und Sensoren
LED	Einzelne LEDs: zeichnen, löschen, Helligkeit
Logik	wenn · dann · sonst
Schleifen	Wiederholungen
Variablen	Werte speichern und verändern

MicroPython: wichtige Befehle

display.show(Image.HEART)	Herz anzeigen
display.set_pixel(2, 2, 9)	LED in der Mitte einschalten
button_a.is_pressed()	Taste A prüfen
sleep(1000)	1 Sekunde warten

Wenn etwas nicht funktioniert

- ◯ Stromversorgung prüfen
- ◯ Blöcke richtig verbunden?
- ◯ Programm übertragen?
- ◯ Sensorwerte prüfen